

Ước lượng MAP và bundle adjustment (MAP Estimation and Bundle Adjustment)

PHỤ THUỘC A/01-hình-học-da-thi (phép chiếu, triangulation — nguồn của phần dư); A/02-mô-hình-camera (mô hình đo đầy đủ); A/03-lie-group-va-toi-uu-tren-da-tap (Jacobian 2×6 và 2×3 , toán tử $\boxtimes$ — chương này lắp chúng thành ma trận). Chương tương ứng ở cây quản tính là `.. /IMU_Fusion/` chương 4.

TÓM TẮT

Đây là chương dựng khung mà luận đề trung tâm của cả cây phát biểu: *SLAM, SfM, hiệu chuẩn và localization là một bài toán MAP duy nhất, chỉ khác ở chỗ biến nào được fix*. Nội dung đi từ xác suất tới cài đặt qua bốn chặng. Chặng một: từ Bayes tới MAP tới bình phương tối thiểu có trọng số — một chuỗi tương đương mà mỗi bước dựa trên một giả thiết cụ thể, và biết rõ giả thiết nào ở đâu là điều kiện để biết khi nào khung này không dùng được. Chặng hai: đồ thị nhân tử như ngôn ngữ mô tả bài toán, và vì sao nó là ngôn ngữ đúng — nó làm cấu trúc thừa hiện ra thành hình vẽ. Chặng ba: cấu trúc thừa ấy và phân bù Schur, thứ biến một bài toán có độ phức tạp bậc ba không chạy nổi thành một bài toán chạy thời gian thực; đây là kỹ thuật quan trọng nhất trong chương xét về giá trị thực dụng. Chặng bốn: marginalization và hàm mất mát bền, hai thứ cần để bài toán sống được với dữ liệu thật và với thời gian vô hạn. Nếu chỉ nhớ một điều: *cấu trúc thừa không phải một tối ưu hoá, nó là lý do bài toán giải được* — và nó đến từ một sự kiện tầm thường, rằng mỗi quan sát chỉ chạm đúng một tư thế và một điểm.

KIẾN THỨC CẦN CÓ TRƯỚC

- **MAP (maximum a posteriori)** — ước lượng chọn giá trị biến làm cực đại xác suất hậu nghiệm $p(\theta | \mathbf{z})$. Khác với hợp lý cực đại (MLE) ở chỗ nó dùng cả tiên nghiệm $p(\theta)$; khi tiên nghiệm đều thì hai cái trùng nhau.
- **phần dư (residual)** — hiệu giữa phép đo thực tế và phép đo mà mô hình dự đoán, $\mathbf{r} = \mathbf{z} - h(\theta)$. Bài toán là làm cho tập phần dư nhỏ nhất theo một nghĩa có trọng số.
- **ma trận thông tin (information matrix)** — nghịch đảo của hiệp phương sai, $\Omega = \Sigma^{-1}$. Dùng làm trọng số: phép đo càng chính xác thì thông tin càng lớn nên càng được tin.
- **whitening** — phép biến đổi $\mathbf{r} \mapsto \Omega^{1/2} \mathbf{r}$ làm mọi phần dư có hiệp phương sai đơn vị, để tổng bình phương thường trở thành tổng bình phương có trọng số đúng. Sau whitening, mọi phần dư so sánh được với nhau bất kể đơn vị gốc.
- **đồ thị nhân tử (factor graph)** — đồ thị hai phía: một bên là biến, một bên là nhân tử (mỗi nhân tử là một số hạng của hàm mục tiêu), cạnh nối nhân tử với các biến nó chạm. Cấu trúc thừa của bài toán hiện ra trực tiếp thành cấu trúc đồ thị.
- **phân bù Schur (Schur complement)** — kỹ thuật khử một khối biến khỏi hệ tuyến tính bằng phép biến đổi đại số, thu được hệ nhỏ hơn chỉ chứa khối còn lại. Trong BA, ta khử các điểm để còn lại hệ chỉ chứa tư thế.
- **marginalization** — loại một biến khỏi bài toán theo nghĩa xác suất — tích phân nó ra — thay vì chỉ xoá nó. Khác biệt là marginalization *giữ lại* thông tin mà biến ấy mang, dưới dạng một nhân tử mới nối các biến còn lại.
- **M-estimator** — họ ước lượng thay bình phương r^2 bằng một hàm $\rho(r)$ tăng chậm hơn ở đuôi, để một phần dư rất lớn (outlier) không kéo nghiệm đi. Huber, Cauchy, Tukey là ba đại diện với ba mức quyết liệt khác nhau.

CÂU HỎI MỞ ĐẦU

- Q1. Bundle adjustment với 1000 tư thế và 100 000 điểm có hơn ba trăm nghìn ẩn. Phân tích Cholesky một ma trận đặc cỡ đó cần khoảng 10^{16} phép toán — không khả thi. Vậy vì sao nó chạy được trong vài giây, và “mẹo” nằm ở đâu?
- Q2. Bình phương tối thiểu là nghiệm MAP *chỉ khi* nhiều là Gauss. Nhưng nhiều trong SLAM không Gauss chút nào — khớp điểm sai tạo ra outlier với phần dư hàng trăm pixel. Vậy vì sao mọi người vẫn dùng bình phương tối thiểu, và cái giá là gì?
- Q3. Cửa sổ trượt phải bỏ bớt trạng thái cũ để giữ kích thước không đổi. Vì sao *xoá* một tư thế cũ khác hẳn *marginalize* nó, và điều gì bị mất khi chọn cách thứ nhất?
- Q4. Levenberg-Marquardt thêm một số hạng λI vào đường chéo Hessian. Giải thích ý nghĩa hình học của số hạng đó, và nói vì sao nó cứu được bộ giải ở chính những chỗ mà A/01 gọi là cấu hình suy biến.
- Q5. Marginalization biến một bài toán thừa thành ít thừa hơn — nó tạo ra fill-in. Vì sao điều đó xảy ra, và vì sao nó là lý do sâu xa khiến cửa sổ trượt không thể mở rộng vô hạn?

1.1 Đặt vấn đề

Ba chương trước cho ta mọi mảnh: hình học nối hai ảnh ràng buộc nhau thế nào, mô hình camera nối tia thành pixel ra sao, Lie group cho công cụ đạo hàm. Còn thiếu đúng một thứ — cách *ghép tất cả lại*.

Bài toán gốc phát biểu như sau. Ta có hàng nghìn khung hình, hàng trăm nghìn điểm bản đồ, hàng triệu quan sát. Mỗi quan sát nói một điều nhỏ và không chắc chắn: “landmark j xuất hiện ở pixel này trong khung i ”. Không quan sát nào đủ để xác định gì; mọi quan sát đều có nhiễu; và chúng *mâu thuẫn* nhau, theo nghĩa không tồn tại cấu hình nào thoả mãn tất cả một cách chính xác. Câu hỏi: cấu hình tư thế và cấu trúc nào là *tốt nhất*, và “tốt nhất” nghĩa là gì?

Ai gặp bài toán này? Nó vượt xa robot học. Trắc địa ảnh gọi nó là bundle adjustment và đã giải nó từ những năm 1950; thiên văn học giải bài toán cùng dạng khi khớp quỹ đạo; địa vật lý giải nó khi định vị tâm chấn. Thực tế đây là một trong những bài toán được nghiên cứu kỹ nhất trong toán ứng dụng, và Triggs cùng cộng sự [1] viết một tổng hợp vẫn là tài liệu tham chiếu chuẩn sau hơn hai mươi năm.

Trước khi có khung MAP, có hai cách tiếp cận và cả hai đều thiếu sót theo cách đáng học. Cách thứ nhất là *ghép cặp rồi nối chuỗi*: dùng hình học epipolar cho từng cặp khung liền kề, rồi nối các phép biến đổi tương đối lại. Nó đơn giản, chạy được, và hỏng vì sai số tích lũy đơn điệu — không có cơ chế nào để một quan sát muộn chỉnh lại ước lượng sớm. Đây chính là *visual odometry*, và nó hữu ích, nhưng nó không phải SLAM vì chữ M — mapping — đòi hỏi một bản đồ nhất quán toàn cục.

Cách thứ hai là *bộ lọc*: giữ một trạng thái và một hiệp phương sai, cập nhật tuần tự mỗi khi có phép đo mới. EKF-SLAM đi theo hướng này và thống trị lĩnh vực tới khoảng 2010. Nó hỏng vì hai lý do mà chương 9 sẽ nói kỹ: chi phí bậc hai theo số landmark, và bất nhất do tuyến tính hoá một lần rồi không bao giờ sửa lại.

Khung MAP giải quyết cả hai, và nó làm thế bằng cách phát biểu bài toán đúng: *tìm cấu hình biến làm cực đại xác suất hậu nghiệm, cho toàn bộ dữ liệu cùng lúc*. Không có thứ tự thời gian đặc quyền, không có thông tin nào bị vứt đi sau khi dùng, và mọi phép đo đều được tuyến tính hoá lại tại ước lượng tốt nhất hiện có. Cái giá là chi phí tính toán — và toàn bộ mục 5 của chương dành cho việc cái giá ấy hoá ra thấp hơn nhiều so với vẻ ngoài, nhờ cấu trúc thưa.

Có một lý do thứ hai để chương này quan trọng, và nó là lý do chính đáng nhất. Khung MAP là chỗ luận đề trung tâm của cây được phát biểu chính xác. Một khi bài toán đã ở dạng “cực tiểu

hoá tổng phần dư có trọng số trên một tập biến”, thì việc phân biệt SLAM với hiệu chuẩn với localization rút xuống thành một câu hỏi duy nhất: *tập biến gồm những gì, và cái nào bị fix?* Bảng ở 00-map là bảng đó, và chương này là chỗ nó có nghĩa toán học.

TRỰC GIÁC

Hình dung mỗi phép đo là một cái lò xo. Một quan sát “landmark j ở pixel này trong khung i ” là một lò xo nối *hai* vật: tư thế khung i và vị trí điểm j . Lò xo có độ cứng tỉ lệ với độ tin cậy của phép đo — đo chính xác thì lò xo cứng.

Bài toán là tìm vị trí cân bằng của cả hệ. Không lò xo nào được thoả mãn hoàn toàn, nhưng tổng năng lượng đàn hồi đạt cực tiểu ở đúng một cấu hình, và cấu hình đó là nghiệm MAP. Ba điều đọc được ngay từ hình ảnh này, và cả ba là nội dung của ba mục sau.

Thứ nhất, *độ thưa*: mỗi lò xo chỉ nối hai vật, không nối tất cả. Với hàng triệu lò xo và hàng trăm nghìn vật, ma trận mô tả hệ gần như toàn số không — và đó là thứ làm bài toán giải được.

Thứ hai, *hướng phẳng*: nếu có một cách dịch chuyển cả hệ mà không lò xo nào căng thêm, thì hệ có vô số vị trí cân bằng. Đó là gauge, và A/05 nói về nó.

Thứ ba, *outlier*: một lò xo bị lắp sai chỗ — nối tư thế i với một điểm không phải điểm nó nhìn thấy — sẽ kéo cả hệ lệch đi. Với lò xo tuyến tính, một lò xo lắp sai và căng rất mạnh có thể thắng hàng trăm lò xo đúng. Mục 6 là về việc làm cho lò xo *mềm dần khi căng quá mức*, để một cái lắp sai không phá cả hệ.

1.2 Từ Bayes tới bình phương tối thiểu

Chuỗi tương đương này ngắn nhưng mỗi bước dựa trên một giả thiết, và biết giả thiết nào ở đâu là điều phân biệt người dùng công cụ với người hiểu công cụ.

A. Ba bước

Theo Bayes, hậu nghiệm tỉ lệ với tích của hợp lý và tiên nghiệm:

$$p(\boldsymbol{\theta} \mid \mathbf{z}) \propto p(\mathbf{z} \mid \boldsymbol{\theta}) p(\boldsymbol{\theta}).$$

Giả thiết thứ nhất nằm ở bước tiếp theo: các phép đo *độc lập* với nhau khi đã biết $\boldsymbol{\theta}$, nên hợp lý tách thành tích:

$$p(\mathbf{z} \mid \boldsymbol{\theta}) = \prod_k p(\mathbf{z}_k \mid \boldsymbol{\theta}).$$

Giả thiết này đáng để ý vì nó bị vi phạm thường xuyên hơn người ta nghĩ: hai đặc trưng cạnh nhau trên cùng một góc ảnh chịu chung một sai số hiệu chuẩn cục bộ, nên sai số của chúng tương quan. Hệ quả là bài toán *đếm thừa* thông tin và hiệp phương sai đầu ra quá lạc quan — một trong các nguồn của bệnh quá tự tin ở A/06.

Lấy $-\log$ biến tích thành tổng và biến cực đại thành cực tiểu:

$$\hat{\boldsymbol{\theta}} = \arg \min_{\boldsymbol{\theta}} \left[- \sum_k \log p(\mathbf{z}_k \mid \boldsymbol{\theta}) - \log p(\boldsymbol{\theta}) \right].$$

Giả thiết thứ hai: nhiễu Gauss, $\mathbf{z}_k = h_k(\boldsymbol{\theta}) + \boldsymbol{\varepsilon}_k$ với $\boldsymbol{\varepsilon}_k \sim \mathcal{N}(\mathbf{0}, \Sigma_k)$. Khi đó $-\log p(\mathbf{z}_k \mid \boldsymbol{\theta})$ bằng $\frac{1}{2} \mathbf{r}_k^\top \Sigma_k^{-1} \mathbf{r}_k$ cộng một hằng số không phụ thuộc $\boldsymbol{\theta}$, và ta được

$$\hat{\boldsymbol{\theta}} = \arg \min_{\boldsymbol{\theta}} \frac{1}{2} \sum_k \mathbf{r}_k(\boldsymbol{\theta})^\top \Omega_k \mathbf{r}_k(\boldsymbol{\theta}), \quad \Omega_k = \Sigma_k^{-1}. \quad (1.1)$$

Đây là bài toán bình phương tối thiểu phi tuyến có trọng số. Tôi tự dẫn lại toàn bộ (cửa a) và đối chiếu [2, §4] cùng [3, §11] (cửa c).

CẦN NHỚ

Chuỗi tương đương này dựa trên **hai giả thiết**, và cả hai đều bị vi phạm trong SLAM thật. Biết chúng ở đâu là biết khi nào khung này nói dối.

Độc lập bị vi phạm khi các phép đo chia nhau một nguồn sai số chung — sai số hiệu chuẩn, biến dạng cục bộ của ống kính, một đối tượng động mà nhiều đặc trưng cùng bám vào. Hệ quả: thông tin bị đếm thừa, hiệp phương sai đầu ra quá nhỏ.

Gauss bị vi phạm mạnh hơn nhiều, và theo hướng nguy hiểm hơn. Khớp điểm sai không tạo ra một phần dư “hơi lớn” mà tạo ra một phần dư *hàng trăm pixel*. Phân bố thật có đuôi rất nặng, trong khi Gauss giả định đuôi mỏng theo hàm mũ. Vì hàm mục tiêu bình phương, một phần dư gấp 100 lần đóng góp gấp 10 000 lần — *một outlier có thể thắng mười nghìn* quan sát đúng.

Đây là lý do mục 6 tồn tại, và là lý do không hệ SLAM thật nào dùng bình phương tối thiểu thuần túy. Nhưng cần nói rõ một điều: hàm mất mát bền *không* sửa giả thiết Gauss cho đúng — nó chỉ hạn chế thiệt hại. Ước lượng thu được không còn là MAP của mô hình nào cả, trừ khi ta diễn giải nó là MAP của một mô hình nhiễu có đuôi nặng, và cách diễn giải ấy chính là đối ngẫu Black-Rangarajan [4].

B. Gauss-Newton và hệ chuẩn

Bài toán (1.1) phi tuyến vì h_k phi tuyến. Gauss-Newton giải bằng cách tuyến tính hoá phần dư quanh ước lượng hiện tại. Với δ là nhiễu loạn theo nghĩa A/03:

$$\mathbf{r}_k(\boldsymbol{\theta} \boxplus \boldsymbol{\delta}) \approx \mathbf{r}_k(\boldsymbol{\theta}) + H_k \boldsymbol{\delta}, \quad H_k = \frac{\partial \mathbf{r}_k}{\partial \boldsymbol{\delta}}.$$

Thay vào và lấy đạo hàm theo $\boldsymbol{\delta}$ rồi cho bằng không:

$$\underbrace{\left(\sum_k H_k^\top \Omega_k H_k \right)}_{\Lambda} \boldsymbol{\delta} = - \sum_k H_k^\top \Omega_k \mathbf{r}_k. \quad (1.2)$$

Đây là **hệ chuẩn**, và Λ là ma trận thông tin của bài toán. Ba điều đáng ghi. Thứ nhất, Λ là *xấp xỉ* Hessian — Hessian thật còn một số hạng chứa đạo hàm bậc hai của h_k , và Gauss-Newton bỏ nó đi; xấp xỉ này tốt khi phần dư nhỏ, và tệ khi phần dư lớn, tức chính lúc có outlier. Thứ hai, Λ luôn nửa xác định dương vì nó là tổng các $H^\top \Omega H$; nó xác định dương *trừ khi* có hướng $\mathbf{u}$ với $H_k \mathbf{u} = 0$ cho mọi k — đó chính xác là định nghĩa hướng không quan sát được ở A/05. Thứ ba, Λ là nghịch đảo của hiệp phương sai nghiệm, nội dung của A/06.

Levenberg–Marquardt thêm damping: $(\Lambda + \lambda D) \boldsymbol{\delta} = -\mathbf{g}$, với D thường là $\text{diag}(\Lambda)$. Ý nghĩa hình học đáng nói rõ vì nó trả lời câu hỏi mở đầu thứ tư. Khi $\lambda \rightarrow 0$ ta có Gauss-Newton thuần, bước dài và nhanh nhưng chỉ đúng khi xấp xỉ tuyến tính đúng. Khi $\lambda \rightarrow \infty$ bước tiến về hướng gradient âm và rất ngắn — chậm nhưng an toàn. Quan trọng hơn cả: λD làm *mọi* hướng có độ cong dương, kể cả hướng mà Λ suy biến. Nghĩa là LM *vẫn chạy* trong cấu hình suy biến — nó trả về bước đi ngắn theo hướng không quan sát được thay vì nổ. Đây là tin tốt cho độ ổn định và là tin xấu cho chẩn đoán: nó biến một lỗi ồn ào thành một lỗi im lặng, đúng như A/05 cảnh báo.

1.3 Đồ thị nhân tử: ngôn ngữ đúng để mô tả bài toán

Hệ chuẩn (1.2) đúng nhưng nó giấu mất cấu trúc. Đồ thị nhân tử là cách viết cùng bài toán sao cho cấu trúc hiện ra.

Đồ thị có hai loại nút. *Nút biến* là thứ ta muốn ước lượng: tư thế, điểm bản đồ, tham số hiệu chuẩn, độ lệch IMU. *Nút nhân tử* là một số hạng của hàm mục tiêu; mỗi nhân tử nối với đúng những biến mà phần dư của nó phụ thuộc vào.

Trong bundle adjustment thuần thị giác, mỗi quan sát “điểm j thấy ở khung i ” là một nhân tử nối đúng hai nút: tư thế T_i và điểm P_j . Không nối gì khác. Đây là sự kiện quan trọng nhất trong cả chương, và mục 5 sẽ khai thác nó tới tận cùng.

Giá trị của cách biểu diễn này có bốn mặt, và mặt thứ tư là mặt quan trọng nhất cho cây này.

Một — độ thưa hiện ra thành hình. Bậc của một nút biến bằng số khối khác không trong hàng tương ứng của Λ . Nhìn đồ thị là biết ma trận thưa đến đâu.

Hai — module hoá. Thêm một loại cảm biến nghĩa là thêm một loại nhân tử, không phải viết lại bộ giải. Đây là lý do thư viện như GTSAM [5] tổ chức quanh khái niệm này, và là lý do C/15 có thể nói về hợp nhất đa cảm biến bằng một câu: *thêm factor vào graph, với hiệp phương sai đúng và công loại outlier riêng cho từng nguồn*.

Ba — thuật toán đồ thị dùng được. Thứ tự khử biến để giảm fill-in là một bài toán đồ thị (A/04 mục 5D); tính liên thông của đồ thị liên quan trực tiếp tới observability.

Bốn — luận đề trung tâm trở nên hiển nhiên. Khi bài toán đã là một đồ thị, “fix một biến” đơn giản là không đặt nó vào tập biến, và “thả một biến” là đặt nó vào. Hiệu chuẩn ngoại tuyến, SLAM, VIO có hiệu chuẩn trực tuyến — ba đồ thị có cùng loại nhân tử, khác nhau ở danh sách nút biến. Không có gì trong bộ giải phải thay đổi.

Bảng 1.1. Cùng một đồ thị nhân tử, bốn bài toán. Chỉ cột “nút biến” thay đổi; loại nhân tử và bộ giải giữ nguyên. Đây là luận đề trung tâm của cây viết ra dưới dạng bảng.

Bài toán	Nút biến	Nhân tử	Hằng số
Hiệu chuẩn từ target	Tư thế bảng, nội, méo	Chiều góc bảng	Toạ độ 3D góc bảng
Bundle adjustment	Tư thế, điểm	Chiều	Nội, méo
Localization	Tư thế khung hiện tại	Chiều	Nội, méo, toàn bộ bản đồ
VIO + calib trực tuyến	Tư thế, vận tốc, độ lệch, điểm, nội, ngoại, t_d	Chiều, tiền tích phân IMU	Không gì

1.4 Độ thưa và phần bù Schur

Đây là mục có giá trị thực dụng cao nhất của chương, và là câu trả lời cho câu hỏi mở đầu thứ nhất.

A. Vì sao bài toán trông không giải được

Với m tư thế và n điểm, số ẩn là $6m + 3n$. Với $m = 1000$ và $n = 100\,000$, đó là 306\,000 ẩn. Phân tích Cholesky một ma trận đặc cỡ N tốn khoảng $N^3/3$ phép toán; với $N = 3 \times 10^5$ đó là cỡ 10^{16}

phép toán, tức nhiều giờ hoặc nhiều ngày trên CPU thông thường. Bộ nhớ còn tệ hơn: ma trận đặc cỡ đó cần khoảng 750 GB ở độ chính xác kép.

Kết luận nếu dừng ở đây: bundle adjustment không khả thi. Nhưng nó chạy trong vài giây, nên có gì đó sai trong lập luận — và cái sai là giả định ma trận đặc.

B. Cấu trúc thưa đến từ đâu

Quay lại sự kiện ở mục 3: mỗi quan sát chỉ chạm *một* tư thế và *một* điểm. Hệ quả trực tiếp lên Λ : khối ứng với cặp (tư thế i , tư thế i') bằng không khi $i \neq i'$ — không quan sát nào chạm hai tư thế cùng lúc. Khối ứng với cặp (điểm j , điểm j') cũng bằng không khi $j \neq j'$, vì lý do tương tự.

Viết Λ theo khối với thứ tự (tư thế trước, điểm sau):

$$\Lambda = \begin{pmatrix} U & W \\ W^\top & V \end{pmatrix},$$

trong đó U cỡ $6m \times 6m$ là khối chéo với các khối 6×6 ; V cỡ $3n \times 3n$ là khối chéo với các khối 3×3 ; và W cỡ $6m \times 3n$ thưa, chỉ khác không ở cặp (i, j) mà khung i thật sự thấy điểm j .

Tính chất quyết định là V *khối chéo* với khối 3×3 , nên nghịch đảo của nó là nghịch đảo từng khối 3×3 độc lập — chi phí tuyến tính theo n và song song hoá được hoàn hảo. Đây là món quà mà cấu trúc bài toán tặng, và toàn bộ mục này là việc nhận món quà đó.

C. Phần bù Schur

Viết hệ chuẩn theo khối, với δ_c là nhiễu loạn tư thế và δ_p là nhiễu loạn điểm:

$$\begin{pmatrix} U & W \\ W^\top & V \end{pmatrix} \begin{pmatrix} \delta_c \\ \delta_p \end{pmatrix} = \begin{pmatrix} \mathbf{g}_c \\ \mathbf{g}_p \end{pmatrix}.$$

Nhân hàng dưới với WV^{-1} rồi trừ khỏi hàng trên, khối W triệt tiêu:

$$(U - WV^{-1}W^\top) \delta_c = \mathbf{g}_c - WV^{-1}\mathbf{g}_p. \quad (1.3)$$

Ma trận $S = U - WV^{-1}W^\top$ là **phần bù Schur**, cỡ $6m \times 6m$ — chỉ phụ thuộc số tư thế, hoàn toàn không phụ thuộc số điểm. Giải (1.3) ra δ_c , rồi thế ngược lấy $\delta_p = V^{-1}(\mathbf{g}_p - W^\top \delta_c)$ — bước thế ngược cũng tuyến tính theo n .

Đếm lại chi phí với $m = 1000$, $n = 100\,000$. Nghịch đảo V : 100 000 phép nghịch đảo ma trận 3×3 , rẻ và song song. Dựng S : cỡ 6000×6000 , tốn công nhưng S cũng thưa — nó chỉ khác không ở cặp tư thế *cùng nhìn thấy ít nhất một điểm chung*. Đây là chỗ khái niệm **đồ thị đồng quan sát** (covisibility graph) ở C/13 có nguồn gốc toán học: nó *chính là* cấu trúc khác không của S . Giải hệ 6000×6000 thưa: trong tầm vài trăm mili giây.

CẦN NHỚ

Phần bù Schur đưa chi phí từ khoảng $O((6m + 3n)^3)$ xuống khoảng $O(m^3) + O(n)$ khi $n \gg m$ — và trong SLAM thì n luôn lớn hơn m hàng trăm lần. Đây **không** phải một tối ưu hoá thêm vào; nó là lý do bundle adjustment tồn tại được như một kỹ thuật thực dụng [1, §6].

Ba hệ quả đáng nhớ, và cả ba xuất hiện lại ở các chương sau.

Một: ma trận S chính là đồ thị đồng quan sát. Hai tư thế nối với nhau trong S đúng khi chúng cùng thấy một điểm. Khái niệm covisibility ở C/13 không phải một heuristic mà là cấu trúc khác không của một ma trận.

Hai: số điểm gần như miễn phí, số tư thế thì không. Điều này định hình toàn bộ kiến trúc SLAM: người ta giữ hàng trăm nghìn landmark nhưng rất tiết kiệm keyframe. Chính sách chọn và loại keyframe ở C/13 có gốc rễ ở đây.

Ba: kỹ thuật này chỉ hoạt động khi V khối chéo. Nếu có một nhân tử nối hai điểm với nhau — chẳng hạn ràng buộc rằng hai điểm nằm trên cùng một mặt phẳng — thì V không còn khối chéo và toàn bộ lợi thế biến mất. Đây là cái giá ít được nói tới của các ràng buộc cấu trúc ở B/08.

D. Thứ tự khử và fill-in

Giải hệ thưa (1.3) bằng Cholesky, nhưng Cholesky sinh ra **fill-in**: các phần tử vốn bằng không trở thành khác không trong thừa số. Lượng fill-in phụ thuộc mạnh vào thứ tự khử biến, và chênh lệch giữa thứ tự tốt và tệ có thể là nhiều bậc độ lớn.

Tìm thứ tự tối ưu là bài toán NP-khó, nên người ta dùng heuristic. **AMD** (approximate minimum degree) và **COLAMD** là hai lựa chọn chuẩn; mọi thư viện thưa tử tế đều cài sẵn. Điểm cần nhớ không phải chi tiết thuật toán mà là: thứ tự biến là một lựa chọn có hậu quả lớn, và mặc định của thư viện thường không tối ưu cho bài toán có cấu trúc đặc biệt như SLAM. Với bài toán có cấu trúc thời gian rõ ràng — tư thế nối tiếp nhau — thứ tự theo thời gian thường tốt hơn thứ tự tổng quát, và đó là một trong những lý do iSAM2 [6] nhanh: nó khai thác cấu trúc ấy một cách tường minh qua Bayes tree.

1.5 Marginalization: sống với thời gian vô hạn

Full batch BA giải mọi thứ cùng lúc, và nó là nghiệm tốt nhất có thể. Nó cũng bất khả thi cho hệ chạy liên tục: số tư thế tăng vô hạn. Cửa sổ trượt giữ kích thước cố định bằng cách loại bớt trạng thái cũ, và cách loại là chỗ quyết định.

A. Xoá so với marginalize

Xoá một tư thế cũ nghĩa là vứt nó cùng mọi nhân tử chạm nó. Rẻ, đơn giản, và sai: thông tin mà các quan sát ấy mang về các biến còn lại bị vứt theo. Hệ quả là ước lượng kém chính xác hơn và — nghiêm trọng hơn — hiệp phương sai bị phóng đại, vì bài toán quên rằng nó từng biết nhiều hơn.

Marginalize nghĩa là tích phân biến ấy ra khỏi phân bố hậu nghiệm. Với phân bố Gauss, phép tích phân này có dạng đóng và nó chính là phần bù Schur — cùng một công thức (1.3), dùng cho mục đích khác. Kết quả là một nhân tử prior mới nối tất cả các biến từng nối với biến bị loại.

Sự trùng nhau giữa hai phép toán đáng dừng lại một câu, vì nó không phải trùng hợp: cả hai đều là “khử một khối biến khỏi hệ tuyến tính trong khi giữ nguyên ảnh hưởng của nó lên phần còn lại”. Trong BA ta khử điểm để giải nhanh rồi thế ngược lấy lại chúng; trong marginalization ta khử tư thế cũ và không lấy lại. Khác biệt duy nhất là ta có quay lại hay không.

B. Cái giá: fill-in

Marginalization có một cái giá mà mọi hệ cửa sổ trượt phải trả, và nó trả lời câu hỏi mở đầu thứ năm.

Khi khử một tư thế nối với k landmark, nhân tử prior mới nối *tất cả* k landmark đó với nhau. Một nút bậc k biến thành một khối đặc $k \times k$. Đồ thị vốn thưa trở nên đặc dần ở vùng biên của cửa sổ, và ma trận thông tin mất cấu trúc khối chéo mà mục 4 dựa vào.

Đây là lý do *sâu xa* khiến cửa sổ trượt không mở rộng vô hạn được: không phải vì số biến lớn, mà vì chi phí của khối prior đặc tăng theo bình phương số biến còn lại nối với nó. Hai cách xử lý trong thực tế. Cách thứ nhất: *chọn khử cái gì*. VINS-Mono marginalize khung cũ nhất khi khung mới là keyframe, và marginalize khung mới thứ hai khi không phải — lựa chọn đó giữ cho prior không nối quá nhiều biến [7]. Cách thứ hai: *làm thưa lại* prior bằng cách xấp xỉ khối đặc bằng một cấu trúc thưa hơn, chấp nhận mất một ít thông tin để đổi lấy chi phí.

C. Cái bẫy nhất quán

Còn một cái giá thứ hai, tinh vi hơn và tổn kém hơn. Nhân tử prior được tính bằng cách tuyến tính hoá tại ước lượng *lúc marginalize*. Các biến nó nối tiếp tục được tối ưu và di chuyển. Nhưng prior thì đóng băng — nó vẫn mang Jacobian của một điểm tuyến tính hoá cũ.

Hậu quả: khi cùng một biến được tuyến tính hoá ở hai điểm khác nhau trong hai phần khác nhau của bài toán, hệ thống có thể *tự tạo ra thông tin* theo một hướng đáng lẽ không quan sát được. Thông tin ấy không đến từ phép đo nào cả; nó là hiện vật của việc tuyến tính hoá không nhất quán. Hệ quả là hiệp phương sai co lại một cách sai trái, tức bộ ước lượng trở nên quá tự tin [8].

Cách chữa chuẩn là **FEJ** (First-Estimates Jacobian): cố định điểm tuyến tính hoá của các biến bị chạm bởi prior. Cây `.. /IMU_Fusion/` chương 5 và chương 8 nói kỹ về nó; ở đây điều cần nhớ là *marginalization không miễn phí về mặt nhất quán*, và một hệ cửa sổ trượt không xử lý chuyện này sẽ báo cáo hiệp phương sai nhỏ hơn sự thật một cách có hệ thống.

I 1.6 Hàm mất mát bền: sống với dữ liệu thật

A. Vì sao bình phương tối thiểu vỡ

Từ khối *Cần nhớ* ở mục 2: hàm mục tiêu bình phương làm một phần dư gấp 100 lần đóng góp gấp 10 000 lần. Một khớp điểm sai — và với bất kỳ front-end nào, tỉ lệ khớp sai vài phần trăm là chuyện bình thường — có thể kéo nghiệm đi xa tùy ý.

Có hai tuyến phòng thủ và chúng bổ sung nhau chứ không thay thế nhau. Tuyến một là *loại bỏ trước khi tối ưu*: RANSAC [9] và các biến thể ở B/07. Tuyến hai là *chịu đựng trong khi tối ưu*: hàm mất mát bền, nội dung của mục này. Cần cả hai vì RANSAC không bắt hết — đặc biệt là các outlier “vừa phải”, có phần dư đủ nhỏ để lọt ngưỡng nhưng đủ lớn để gây hại.

B. M-estimator và IRLS

Thay $\frac{1}{2}r^2$ bằng $\rho(r)$ tăng chậm hơn ở đuôi. Ba lựa chọn chuẩn:

Huber: bậc hai cho $|r| \leq \delta$, tuyến tính ngoài đó. Lỗi, nên không tạo cực tiểu địa phương mới — tính chất này quan trọng và là lý do Huber là mặc định an toàn.

Cauchy: $\rho(r) = \frac{c^2}{2} \log(1 + r^2/c^2)$. Không lỗi, giảm ảnh hưởng outlier mạnh hơn, nhưng có thể tạo cực tiểu địa phương.

Tukey: bảo hoà hoàn toàn — phần dư quá ngưỡng đóng góp *đúng bằng không*. Quyết liệt nhất, và đòi khởi tạo tốt nhất.

Cài đặt qua **IRLS** (iteratively reweighted least squares): ở mỗi vòng lặp, gán trọng số $w_k = \rho'(|r_k|)/|r_k|$ rồi giải bài toán bình phương tối thiểu có trọng số thông thường. Với Huber, $w_k = \min(1, \delta/|r_k|)$.

Ví dụ số (cửa b). Giả sử $\delta = 1,5$ px (whitened). Một phần dư $|r| = 1$ px cho $w = 1$ — không bị hạ trọng số. Một phần dư $|r| = 15$ px cho $w = 0,1$. Đóng góp vào hệ chuẩn tỉ lệ với $w r^2$: với bình phương thuần, tỉ số đóng góp giữa hai phần dư là $15^2/1^2 = 225$; với Huber, là $0,1 \times 225 = 22,5$. Ảnh hưởng của outlier giảm mười lần, đúng bằng hệ số $\delta/|r|$. Cấu trúc thừa hoàn toàn không đổi vì trọng số chỉ nhân vào từng khối — đây là lý do M-estimator gần như miễn phí về mặt tính toán, và là lý do không có cơ gì để không dùng nó.

C. GNC: khi khởi tạo quá tệ

Hàm không lồi như Tukey chỉ hoạt động nếu bắt đầu đủ gần nghiệm đúng. Khi không có điều đó — chẳng hạn với loop closure sai ở B/11, nơi một cạnh sai có thể làm gập cả bản đồ — cần một chiến lược khác.

GNC (graduated non-convexity) [10] bắt đầu bằng một hàm mất mát gần lồi rồi *từ từ* làm nó không lồi hơn qua các vòng lặp. Trực giác: giải bài toán dễ trước để có khởi tạo tốt, rồi dùng nó làm khởi tạo cho bài toán khó hơn một chút, lặp lại. Nền lý thuyết là đối ngẫu Black–Rangarajan [4], cho thấy tối ưu với M-estimator tương đương với tối ưu đồng thời trên biến gốc và một tập biến “trọng số outlier” — nên GNC là một dạng làm mềm dần bài toán gán nhãn inlier/outlier.

Giá trị thực dụng của GNC cao nhất ở pose graph có loop closure sai, và đó là lý do nó xuất hiện lại ở B/11 như một trong các kỹ thuật back-end bền.

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Ceres [11]	Schur complement solver, ordering tự động, LossFunction (Huber/Cauchy/Tukey)	Cài đặt tham chiếu của toàn chương; đọc tài liệu phần SPARSE_SCHUR
g2o [12]	Đồ thị nhân tử tường minh, Schur cho cấu trúc BA	Mã ngắn, dễ đọc cấu trúc khối U, W, V ở mục 4B
GTSAM [5]	iSAM2 [6], Bayes tree, khử biến gia tăng	Minh hoạ mục 4D: khai thác thứ tự thời gian để giảm fill-in
ORB-SLAM3 [13]	Local BA trên cửa sổ đồng quan sát; Huber cho mọi cạnh chiếu	S chính là đồ thị đồng quan sát — mục 4C thành kiến trúc
VINS-Mono [7]	Marginalization có chọn lọc khung cũ nhất hay khung mới thứ hai	Mục 5B: chiến lược chọn khử để kiểm soát fill-in
Basalt [14]	Square-root BA — phân tích QR thay vì hệ chuẩn	Ổn định số học tốt hơn; số điều kiện bình phương của Λ là lý do

MINI PROJECT SLAM • Tự viết BA thừa, rồi đo chính xác lợi ích của Schur • 4 ngày

- Mục tiêu** biến “Schur làm nó nhanh hơn” từ một câu nghe được thành một đường cong bạn tự đo.
- Dữ liệu** tự sinh: một quỹ đạo, một đám mây điểm, các quan sát có nhiễu Gauss đã biết σ . Tự sinh là bắt buộc ở bài này vì bạn cần chân trị để đo sai số.
- Các bước** dựng bài toán BA với Jacobian từ A/03; cài *ba* bộ giải trên cùng bài toán — Cholesky đặc, Cholesky thừa không Schur, và Schur + Cholesky thừa; đo thời gian và bộ nhớ theo n từ 100 tới 100 000 điểm với m cố định, rồi theo m với n cố định; thêm 5% outlier và chạy lại với bình phương thuần rồi với Huber; cuối cùng cài marginalization và đo fill-in của khối prior theo số lần khử.

- Xong khi** bạn có ba đường cong thời gian theo n trên trục log-log, và độ dốc của chúng khớp với phân tích ở mục 4C: đặc gần bậc ba, Schur gần tuyến tính. Bạn có một bảng sai số tư thế với và không có Huber ở 5% outlier, cho thấy khác biệt bậc độ lớn. Và bạn có một đường cong mật độ khối prior tăng theo số lần marginalize — xác nhận mục 5B bằng số của chính mình.
- Nổi với** A/05-observability-gauge-va-suy-bien dùng chính ma trận Λ bạn vừa dựng để xem phổ giá trị riêng; A/06-bat-dinh-va-lan-truyen-sai-so lấy Λ^{-1} làm hiệp phương sai; B/09-paradigm-uoc-luong-back-end đặt bộ giải này vào các kiến trúc khác nhau.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Mẹo nằm ở *cấu trúc thưa*, và cụ thể là ở việc khối V — ứng với các điểm — là khối chéo với các khối 3×3 . Lý do: mỗi quan sát chỉ chạm một tư thế và một điểm, nên không có nhân tử nào nối hai điểm với nhau. Phân bù Schur khai thác điều đó để khử toàn bộ $3n$ biến điểm với chi phí tuyến tính theo n , để lại một hệ chỉ cỡ $6m \times 6m$. Chi phí đi từ $O((6m + 3n)^3)$ xuống khoảng $O(m^3) + O(n)$. Với $m = 1000$, $n = 100\,000$, đó là khác biệt giữa nhiều ngày và vài trăm mili giây.
- A2. Vì bình phương tối thiểu có hai thứ mà không lựa chọn nào khác có cả hai: nghiệm dạng đóng cho bài toán tuyến tính hoá, và *bảo toàn cấu trúc thưa*. Một mô hình nhiều đuôi nặng đúng đắn sẽ phá vỡ cả hai. Cái giá là cực nhạy với outlier — phần dư gấp 100 lần đóng góp gấp 10 000 lần. Cách xử lý là hai tuyến: RANSAC loại bỏ trước khi tối ưu, và M-estimator chịu đựng trong khi tối ưu. Nhưng phải nói rõ: M-estimator *không* sửa giả thiết Gauss cho đúng, nó chỉ hạn chế thiệt hại; nghiệm thu được là MAP của một mô hình nhiều đuôi nặng, theo đối ngẫu Black-Rangarajan [4].
- A3. Xoá vứt cả biến lẫn mọi nhân tử chạm nó, nên thông tin mà các quan sát ấy mang về các biến *còn lại* mất theo. Marginalize tích phân biến ra khỏi hậu nghiệm — với Gauss thì đó chính là phân bù Schur — và giữ lại thông tin đó dưới dạng một nhân tử prior mới nối các biến còn lại. Cái mất khi xoá: độ chính xác giảm, và hiệp phương sai bị phóng đại vì bài toán quên rằng nó từng biết nhiều hơn. Cái giá khi marginalize: fill-in (prior là khối đặc) và nguy cơ bất nhất do điểm tuyến tính hoá bị đóng băng, chữa bằng FEJ.
- A4. λD cộng độ cong dương vào *mọi* hướng, kể cả hướng mà Λ suy biến. Hình học: nó biến một cái máng phẳng thành một cái bát nông. Khi $\lambda \rightarrow 0$ ta có Gauss-Newton (bước dài, nhanh, chỉ đúng khi xấp xỉ tuyến tính đúng); khi $\lambda \rightarrow \infty$ ta có gradient descent bước ngắn (chậm, an toàn). Nó “cứu” bộ giải ở cấu hình suy biến theo nghĩa hệ thống không nổ — nhưng đó là tin xấu cho chẩn đoán: nó trả về một bước ngắn theo hướng không quan sát được và biến một lỗi ồn ào thành lỗi im lặng. LM che triệu chứng chứ không chữa bệnh; bệnh thuộc về A/05.
- A5. Vì khi khử một tư thế nối với k landmark, mọi cặp trong k landmark ấy trở nên tương quan — thông tin “chúng cùng được nhìn từ một chỗ” phải được lưu ở đâu đó, và chỗ đó là một khối đặc $k \times k$ trong prior. Đồ thị thưa trở nên đặc dần ở vùng biên cửa sổ, và V mất tính khối chéo mà Schur dựa vào. Đó là lý do sâu xa: không phải số biến lớn, mà là chi phí khối prior đặc tăng theo bình phương. Hai cách xử lý: chọn khử cái gì để prior không nối quá nhiều biến (VINS-Mono), hoặc làm thưa lại prior bằng xấp xỉ.

NÓI LẠI THẬT ĐƠN GIẢN

Tưởng tượng một tấm lưới làm bằng hàng nghìn sợi chun, mỗi sợi nối hai cái đinh ghim trên một tấm bảng. Mỗi sợi chun là một lần cái máy “nhìn thấy” một thứ gì đó; mỗi cái đinh là một chỗ mà máy từng đứng, hoặc một điểm mốc trong phòng. Không sợi chun nào chính xác cả — mỗi sợi hơi dài hoặc hơi ngắn so với chỗ nó cần. Nên không có cách nào đặt các đinh để mọi sợi đều thoải mái. Việc phải làm là tìm chỗ đặt đinh sao cho *tổng* độ căng nhỏ nhất. Đó là toàn bộ bài toán.

Điều tưởng như làm nó bất khả thi: có hàng trăm nghìn đinh. Nhưng có một sự kiện cứu tất cả. Mỗi sợi chun chỉ nối *hai* đinh, và một trong hai luôn là một điểm mốc chứ không phải một chỗ đứng. Nghĩa là không có sợi nào nối điểm mốc này với điểm mốc kia. Các điểm mốc chỉ “nói chuyện” với nhau *gián tiếp*, qua các chỗ đứng.

Nhờ đó, ta có thể giải quyết từng điểm mốc một cách độc lập — rất nhanh, và làm song song được — rồi chỉ còn lại một bài toán nhỏ giữa vài trăm chỗ đứng. Chuyện từ nhiều ngày rút xuống còn một phần giây. Đó là ý tưởng duy nhất đáng nhớ của chương, và nó không phải toán cao siêu mà là việc nhận ra một sự kiện tầm thường rồi khai thác nó triệt để.

Còn một chuyện nữa. Nếu có một sợi chun bị mắc nhầm chỗ — nối một chỗ đứng với một điểm mốc mà nó chưa bao giờ nhìn thấy — thì sợi đó sẽ căng cực mạnh và kéo lệch cả tấm lưới. Cách chữa không phải tìm cho ra sợi hỏng, vì có hàng triệu sợi. Cách chữa là làm cho mọi sợi chun *mềm dần khi bị kéo quá căng*, để không sợi nào một mình thắng được số đông.

Chỗ không rút gọn được: chỗ không rút gọn được là *marginalization*. Ý tưởng “bỏ bớt đỉnh cũ nhưng giữ lại điều ta đã học được từ chúng” nghe đơn giản, nhưng cái giá của nó — các đỉnh còn lại trở nên dính vào nhau, và thông tin cũ được ghi lại ở một toạ độ đã lỗi thời — thì không có hình ảnh nào diễn tả gọn được. Đó là lý do các hệ cửa sổ trượt phức tạp hơn nhiều so với vẽ ngoài, và là lý do B/09 phải viết riêng một chương.

Một câu: Mỗi phép đo chỉ chạm hai thứ — và chính sự nghèo nàn đó làm bài toán không lồ trở nên giải được.

1.7 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *MAP với nhiều Gauss tương đương bình phương tối thiểu có trọng số* — là một suy dẫn toán học thuần tuý dưới hai giả thiết đã nêu rõ, nên nó không bị bác bỏ bởi thực nghiệm mà chỉ bởi lỗi suy dẫn. Cái *có thể* bị bác bỏ là mệnh đề ẩn rằng hai giả thiết ấy đủ gần đúng để khung này dùng được. Nó bị bác bỏ nếu một hệ dùng mô hình nhiều đuôi nặng đúng đắn — chứ không phải M-estimator — cho kết quả tốt hơn đáng kể với chi phí chấp nhận được. Tôi chưa thấy công trình nào chứng minh điều đó trong SLAM thời gian thực, nhưng tôi cũng chưa khảo sát có hệ thống (tôi suy ra, chưa đối chiếu nguồn).

Mệnh đề thứ hai — *Schur đưa chi phí từ bậc ba theo tổng số biến xuống gần tuyến tính theo số điểm* — là kết quả đếm phép toán dưới giả thiết $n \gg m$ và V khối chéo. Nó bị bác bỏ nếu đường cong đo được không có độ dốc như dự đoán, và mini project chính là phép đo. Một trường hợp mà nó *thật sự* không đúng và đáng biết: khi đồ thị đồng quan sát rất đặc — mọi khung nhìn thấy mọi điểm, như trong một quét vòng quanh một vật nhỏ — thì S đặc và lợi thế giảm mạnh.

Mệnh đề thứ ba — *marginalization gây bất nhất nếu không dùng FEJ* — đã được thiết lập trong ngữ cảnh bộ lọc [8]. Điều tôi *chưa* kiểm được là mức độ nghiêm trọng trong ngữ cảnh cửa sổ trượt tối ưu hoá, nơi các biến vẫn được tái tuyến tính hoá trong cửa sổ. Có lý do để nghĩ nó nhẹ hơn trong bộ lọc, nhưng tôi không có số liệu, và đó là một mục trong 99-chưa-biết.md.

1.8 Kết nối

Chương này là trung tâm của Phần A và là chỗ luận đề trung tâm có nghĩa toán học.

Nối lên trên: A/03-lie-group-va-toi-uu-tren-da-tap cung cấp các khối Jacobian 2×6 và 2×3 mà chương này lắp thành ma trận; A/02-mo-hinh-camera cung cấp hàm đo h ; A/01-hinh-hoc-da-thi cung cấp khởi tạo, thứ mà Gauss-Newton bắt buộc phải có.

Nối xuống dưới trong Phần A. A/05-observability-gauge-va-suy-bien lấy chính ma trận Λ ở (1.2) và hỏi khi nào nó suy biến — toàn bộ chương 5 là một câu hỏi về ma trận của chương 4. A/06-bat-dinh-va-lan-truyen-sai-so lấy Λ^{-1} làm hiệp phương sai và hỏi con số đó có đáng tin không.

Nối sang các phần sau. B/09-paradigm-uoc-luong-back-end là chương mở rộng trực tiếp nhất: nó cho thấy bộ lọc, cửa sổ trượt, smoothing gia tăng và full batch đều là cùng

bài toán MAP với các lựa chọn khác nhau về việc marginalize cái gì và tuyến tính hoá lại bao nhiêu lần. B/08-mo-hinh-phan-du thay đổi hàm h — indirect, direct, hay lai — mà không đổi gì trong khung này, và đó là bằng chứng cho tính tổng quát của nó. B/11-loop-closure-va-nhat-quan-toan-cuc dùng GNC và các kỹ thuật bên ở mục 6 để sống với loop closure sai. C/13-quan-ly-ban-do có gốc rễ toán học ở mục 4C: covisibility graph là cấu trúc khác không của S . C/14-hieu-chuan-nhu-mot-nhanh-cua-slam là chương chứng minh Bảng 1.1 bằng cách thực sự đi từ một ô sang ô khác.

Nối ngang: ../IMU_Fusion/ chương 4 trình bày cùng khung MAP từ góc quán tính, và chương 9 ở đó nói kỹ hơn về marginalization trong VIO. Chỗ nào cần chi tiết về tiền tích phân như một nhân tử thì đọc ở đó.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Chuỗi từ Bayes tới bình phương tối thiểu dựa trên hai giả thiết. Nêu cả hai, nêu chỗ chúng bị vi phạm trong SLAM thật, và nêu hậu quả cụ thể của mỗi vi phạm.
2. Vì sao khối V trong Hessian BA là khối chéo, và điều gì trong cấu trúc bài toán đảm bảo điều đó? Nêu một loại ràng buộc sẽ phá vỡ nó.
3. Tự dẫn lại phần bù Schur từ hệ khối 2×2 , không nhìn bài. Rồi đếm chi phí cho $m = 500$, $n = 50\,000$.
4. Vì sao ma trận S chính là đồ thị đồng quan sát, và điều đó có hệ quả gì cho chính sách chọn keyframe?
5. Nêu ba khác biệt giữa xoá và marginalize một tư thế cũ, và nêu cái giá riêng của marginalize.
6. Levenberg-Marquardt giúp bộ giải chạy được trong cấu hình suy biến. Vì sao đó vừa là tin tốt vừa là tin xấu?
7. Huber với $\delta = 1,5$. Tính trọng số cho phần dư 0,5, 3, và 30 pixel, rồi nói ảnh hưởng tương đối của ba quan sát ấy lên hệ chuẩn so với trường hợp bình phương thuần.

Nguồn

- [1] B. Triggs, P. F. McLauchlan, R. I. Hartley, et al., “Bundle Adjustment — A Modern Synthesis,” in *Vision Algorithms: Theory and Practice*, mức tin A; tài liệu tham chiếu chuẩn cho BA, gauge và độ thừa, Springer, 2000, pp. 298–372.
- [2] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017, mức tin A; nguồn chuẩn cho ước lượng trên đa tạp.
- [3] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005, mức tin A.
- [4] M. J. Black and A. Rangarajan, “On the Unification of Line Processes, Outlier Rejection, and Robust Statistics with Applications in Early Vision,” *International Journal of Computer Vision*, vol. 19, no. 1, pp. 57–91, 1996, mức tin A; nguồn của đối ngẫu Black-Rangarajan.
- [5] F. Dellaert and M. Kaess, “Factor Graphs for Robot Perception,” *Foundations and Trends in Robotics*, vol. 6, no. 1-2, pp. 1–139, 2017, mức tin A.
- [6] M. Kaess, H. Johannsson, R. Roberts, et al., “iSAM2: Incremental Smoothing and Mapping Using the Bayes Tree,” *The International Journal of Robotics Research*, vol. 31, no. 2, pp. 216–235, 2012, mức tin A.
- [7] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [8] G. P. Huang, A. I. Mourikis, and S. I. Roumeliotis, “Analysis and Improvement of the Consistency of Extended Kalman Filter Based SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A; nguồn của First-Estimates Jacobian, 2008.
- [9] M. A. Fischler and R. C. Bolles, “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981, mức tin A.
- [10] H. Yang, P. Antonante, V. Tzoumas, et al., “Graduated Non-Convexity for Robust Spatial Perception: From Non-Minimal Solvers to Global Outlier Rejection,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1127–1134, 2020, mức tin A.
- [11] S. Agarwal, K. Mierle, and The Ceres Solver Team, *Ceres Solver*, phần mềm mã nguồn mở, mức tin B; tài liệu phần mềm, không phải công trình phản biện, 2022.
- [12] R. Kümmerle, G. Grisetti, H. Strasdat, et al., “g2o: A General Framework for Graph Optimization,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2011.
- [13] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.
- [14] N. Demmel, C. Sommer, D. Cremers, et al., “Square Root Bundle Adjustment for Large-Scale Reconstruction,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.